

好的，这是一个非常核心且强大的 GAS 功能。UGameplayModMagnitudeCalculation（通常简称为 ModMagCalc 或 MMC）是实现复杂、数据驱动的属性修改的终极工具。您提供的 MMC_MaxHealth 代码是一个完美的、规范的教学范例。

我们将详细分解使用自定义计算类（CustomCalculationClass）修改属性的完整流程，并解释每一步的含义、必要性及其作用。

一、什么是 ModMagCalc (MMC) 及其适用场景？

简单的属性修改（如“+10 生命”）可以直接在 GE 中设置。稍微复杂一点的（如“法力上限 = 智力 * 1.5”）可以使用 AttributeBased 修改器。

但是，当您的计算逻辑满足以下任意一点时，就必须使用 MMC：

- **计算依赖多个属性：**例如，“物理伤害 = (力量 * 2) + (敏捷 * 0.5)”。
- **计算需要非属性数据：**例如，您代码中的“玩家等级”¹。
- **计算包含复杂的、非线性的逻辑：**例如，伤害减免公式可能涉及复杂的曲线或百分比递减。

MMC 允许您编写任意复杂的 C++ 代码，来计算出一个 Modifier 的最终量值（Magnitude）。

二、CustomCalculationClass (MMC) 的规范流程与步骤

整个流程分为两大步：**1. 在 C++ 中创建计算逻辑** 和 **2. 在 UE 编辑器中应用该逻辑**。

第一部分：在 C++ 中创建计算类

步骤 1：创建继承自 UGameplayModMagnitudeCalculation 的类

- **操作：**您需要创建一个新的 C++ 类，继承自 UGameplayModMagnitudeCalculation。在您的代码中，这就是 UMMC_MaxHealth 类²。
- **含义与必要性：**这声明了一个新的、专门用于计算 GE 量值的“计算器”类。继承这个基类是必要的，因为它提供了我们需要重写（override）的核心函数 CalculateBaseMagnitude_Implementation 和 GAS 能够识别的结构。

步骤 2：在构造函数中“声明”需要捕获的属性

- **操作：**在 .cpp 文件的构造函数中，您必须明确定义这个计算器在执行计算时，需要从“源（Source）”或“目标（Target）”身上读取哪些属性。

C++

```
UMMC_MaxHealth::UMMC_MaxHealth()
```

```
{
```

```
    // 1. 定义一个捕获规则
```

```
    VigorDef.AttributeToCapture = UAuraAttributeSet::GetVigorAttribute(); // 要捕获哪个属性?
```

```
    VigorDef.AttributeSource = EGameplayEffectAttributeCaptureSource::Target; // 从谁身上捕获? (目标)
```

```
    VigorDef.bSnapshot = false; // 是否在创建时就“快照”该值? (否, 实时获取)
```

```
    // 2. 将规则添加到列表中
```

```
    RelevantAttributesToCapture.Add(VigorDef);
```

```
}
```

3

- **含义与必要性:**

- **FGameplayEffectAttributeCaptureDefinition (VigorDef):** 这是一个“捕获规则”结构体⁴。它告诉 GAS: “当我这个计算器工作时, 我需要你提前为我准备好某个角色的某个属性值。”
- **AttributeToCapture:** 指定要捕获的具体属性, 这里是 Vigor (精力)⁵。
- **AttributeSource:** 定义从谁身上获取该属性。Target 指效果的接收方, Source 指效果的施加方。这里是根据目标自身的韧性来计算, 所以是 Target⁶。
- **bSnapshot:** 定义何时捕获该属性值。false 表示每次计算都**实时**获取最新值; true 表示在 GE 的 Spec 被创建的那一刻就“拍照”记录下来, 之后不再改变⁷。
- **RelevantAttributesToCapture.Add(VigorDef):** 这是最关键的一步。您必须将定义好的捕获规则添加到 RelevantAttributesToCapture 这个数组中。GAS 系统会在后台查看这个数组, **提前**收集好所有需要的属性数据, 并打包到 FGameplayEffectSpec 中, 以便后续的计算函数能够安全、高效地访问它们。**如果没有这一步, 后续的**

GetCapturedAttributeMagnitude 将无法获取到任何值。⁸

步骤 3：在 CalculateBaseMagnitude_Implementation 中实现核心计算逻辑

- **操作：**这是您必须重写的核心函数，所有计算都在这里发生。它的返回值就是这个 Modifier 的最终数值。

C++

```
float UMMC_MaxHealth::CalculateBaseMagnitude_Implementation(const  
FGameplayEffectSpec& Spec) const  
  
{  
  
    // ...  
  
}
```

9

- **函数内部详解：**

1. 获取 Tags:

C++

```
const FGameplayTagContainer* SourceTag =  
Spec.CapturedSourceTags.GetAggregatedTags();  
  
const FGameplayTagContainer* TargetTag =  
Spec.CapturedTargetTags.GetAggregatedTags();
```

10

- **作用：**从 Spec（当前 GE 实例的详细数据）中获取施法者和目标身上的所有 GameplayTag。这允许您实现基于标签的条件逻辑（例如，“如果目标带有‘燃烧’标签，则伤害加倍”）。

2. 准备求值参数:

C++

```
FAggregatorEvaluateParameters EvaluationParameters;  
  
EvaluationParameters.SourceTags = SourceTag;  
  
EvaluationParameters.TargetTags = TargetTag;
```

11

- **作用:** 创建一个参数包，将刚刚获取的 Tags 传递给属性获取函数。

3. 安全地获取捕获的属性值:

C++

```
float Vigor = 0.f;
```

```
GetCapturedAttributeMagnitude(VigorDef, Spec, EvaluationParameters, Vigor);
```

12

- **作用:** 这是获取您在构造函数中声明的属性值的**唯一正确方法**。您不能直接访问 Target->GetVigor()。
GetCapturedAttributeMagnitude 函数会安全地从 Spec 中解包出之前预先捕获的 Vigor 值，并存入您提供的 Vigor 浮点变量中。

4. 执行自定义逻辑:

C++

```
Vigor = FMath::Max(Vigor, 0.f); // 确保韧性不为负
```

```
ICombatInterface* CombatInterface =  
Cast<ICombatInterface>(Spec.GetContext().GetSourceObject());
```

```
const int32 PlayerLevel = CombatInterface->GetPlayerLevel(); // 获取玩家等级
```

13

- **作用:** 这是 MMC 最强大的地方。您可以执行任何 C++ 代码。在这里，代码首先对获取的 Vigor 值进行了夹值¹⁴。然后，它通过 Spec.GetContext() 获取效果的上下文，并从中拿到 SourceObject（通常是创建此效果的角色或技能），通过接口（ICombatInterface）调用一个非 GAS 的函数 GetPlayerLevel()¹⁵。这展示了 MMC 如何将 GAS 系统与您游戏的其他系统（如等级系统）连接起来。

5. 返回最终计算结果:

C++

```
return 80.f + Vigor * 2.5f + 10.f * PlayerLevel;
```

16

- **作用:** 函数返回的这个 float 值，就是最终应用到 GE 的 Modifier

上的量值。例如，如果这是一个 MaxHealth 的 Add 类型的 Modifier，那么角色的最大生命值就会增加 $80 + \text{Vigor} * 2.5 + \text{PlayerLevel} * 10$ 。

第二部分：在 UE 编辑器中应用 MMC

1. 创建或打开一个 **GameplayEffect** 蓝图。
2. 添加一个新的 Modifier（修改器）。
3. 在 Modifier 的细节面板中，找到 **Magnitude Calculation Type (量值计算类型)**。
4. 将其下拉菜单选为 **Custom Calculation Class (自定义计算类)**。
5. 在下方新出现的 **Calculation Class (计算类)** 属性中，从下拉菜单里选择您刚刚创建的 C++ 类，即 **MMC_MaxHealth**。

完成以上步骤后，这个 Modifier 的 Magnitude 就会完全由您的 `UMMC_MaxHealth::CalculateBaseMagnitude_Implementation` 函数的返回值来决定。